

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1101 Introducción a la
Programación

Documentación técnica

OvercookTec

Estudiantes:

Fabricio Guillén
Acevedo

Abigail Saborío
Jiménez

Profesor:

Leonardo Araya

Martinez

Semestre I 2026

0. Indice

0. Indice.....	2
1. Introducción.....	4
1.1. Propósito del documento.....	4
1.2. Descripción del proyecto.....	4
1.3 Objetivos y Alcance.....	4
1.4 Definiciones, Acrónimos y Abreviaturas.....	4
2. Descripción General del Sistema.....	5
2.1. Perspectiva del Producto.....	5
2.2. Funcionalidades principales.....	5
2.3. Restricciones del sistema, Supuestos y Dependencias.....	5
3. Requisitos del Sistema.....	7
3.1. Requisitos Funcionales.....	7
3.2. Requisitos No Funcionales.....	7
4. Diseño de la Solución.....	8
4.1. Visión General de la Arquitectura.....	8
4.2. Descripción de Componentes.....	8
4.4. Diagrama de Clases y Modelo de Datos.....	9
4.5. Diagrama de Flujo y Secuencia.....	9
4.6. Decisiones de Diseño y Justificación.....	9
5. Implementación y Validación de Calidad.....	11
5.1. Estructura del repositorio.....	11
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución.....	11
5.3. Estrategias y Casos de Prueba.....	12
5.4. Resultados de pruebas y problemas encontrados.....	12
6. Declaración de uso de Inteligencia Artificial.....	14
6.1. Herramientas de IA utilizadas.....	14
6.2. Herramientas de IA utilizadas.....	14
6.3. Declaración de Responsabilidad y Autoría.....	15
7. Conclusión.....	16
8. Bibliografía.....	17

1. Introducción

Este proyecto nació como una forma de aplicar lo visto sobre Programación Orientada a Objetos y la utilización de librerías externas para así evaluar nuestro rendimiento como programadores.

1.1. Propósito del documento

Ejemplo: El presente documento describe la arquitectura, diseño e implementación del sistema proyecto "OvercookTec, destinado a los profesores del curso introducción a la programación.

1.2. Descripción del proyecto

Este programa es una aplicación de escritorio que imita de forma básica las mecánicas del videojuego Overcooked. Con cocinas y platillos que deben prepararse a tiempo para ganar puntos.

1.3 Objetivos y Alcance

El alcance del proyecto es reducido a los evaluadores del mismo.

Objetivo general: Desarrollar un prototipo de un videojuego inspirado en overcooked utilizando la programación orientada a objetos

Objetivos específicos:

1. Utilizar el lenguaje de programación Python, así como estructuras de datos simples para la creación de un juego sencillo
2. Desarrollar conocimiento en la programación orientada a objetos, las clases, atributos, métodos e instancias.
3. Documentar en un medio persistente, aspectos generales del desarrollo del proyecto para su posterior consulta.

2. Descripción General del Sistema

Este programa como se comentó en la introducción es un prototipo muy similar al videojuego Overcooked, en dónde sos un chef que tiene que cocinar platillos en un restaurante y entregar el platillo en el tiempo correspondiente y en el estado óptimo para obtener la mayor puntuación posible.

2.1. Perspectiva del Producto

Este programa opera en una carpeta con varios archivos .py, estos archivos son partes del código, se tomó la decisión de dividir el código para así poder organizar de mejor manera el proyecto. El programa utiliza librerías externas, las librerías externas son: pygame y random.

Pygame se utiliza para desarrollar la interfaz gráfica del videojuego.

Random se utiliza para la generación aleatoria de las órdenes.

2.2. Funcionalidades principales

- **Movimiento e interacción:** Los jugadores se moverán e interactúan en el juego usando las teclas del teclado

2.3. Restricciones del sistema, Supuestos y Dependencias

Restricciones:

- **Tener archivos importantes:** El sistema debe tener los archivos importantes para la correcta ejecución del código.

Supuestos:

- El usuario debe tener instalado python para la correcta ejecución del programa

3. Requisitos del Sistema

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El jugador puede mover al chef con las teclas	Alta	El chef se mueve en las 4 direcciones sin salirse de la pantalla
RF-002	El chef puede recoger ingredientes del almacén	Alta	El chef agarra el ingrediente al presionar la tecla de interacción
RF-003	Las estaciones procesan ingredientes con un timer	Alta	El ingrediente cambia de estado según el tiempo en la estación
RF-004	El juego genera órdenes aleatorias	Alta	Aparecen pedidos de recetas al azar durante el nivel
RF-005	El juego tiene 3 niveles con cocinas distintas	Media	Cada nivel tiene recetas y estaciones diferentes

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El juego debe correr fluidamente	60 FPS constantes
RNF-002	Usabilidad	Los controles deben ser intuitivos	2 jugadores simultáneos sin conflicto de teclas
RNF-003	Disponibilidad	El juego debe correr en Windows	Compatible con Python 3.x y pygame 2.x

4. Diseño de la Solución

4.1. Visión General de la Arquitectura

El sistema sigue el patrón MVC. El Modelo son las clases de entities/ que guardan la lógica del juego. La Vista es el módulo ui/ que dibuja todo en pantalla. El Controlador es game.py que coordina los eventos, actualizaciones y el dibujado

4.2. Descripción de Componentes

Describe cada componente o módulo del sistema: qué hace, qué tecnología usa, qué interfaces expone y qué dependencias tiene.

Ejemplo:

Componente	Responsabilidad	Tecnología	Dependencias
Player.py	Controla el movimiento e interacción de los chefs	Python 3.x, Pygame	constants.py
Station	Gestiona las estaciones de cocina y sus timers	Python 3.x, Pygame	constants.py , ingrediente.py
Recipe	Define las recetas y verifica si están completas	Python 3.x	constants.py
Order	Genera pedidos aleatorios con timer de expiración	Python 3.x, random	recipe.py
ingrediente.py	Representa un ingrediente con su estado y estación requerida	Python 3.x	Ninguna
level.py	Define estaciones y recetas disponibles por nivel	Python 3.x, Pygame	recipe.py , station.py , constants.py
game.py	Coordina el game loop, eventos, actualización y dibujado	Python 3.x, Pygame	player.py , level.py , order.py , constants.py
constants.py	Almacena constantes globales, teclas y diccionario de recetas	Python 3.x, Pygame	Pygame

4.3. Stack Tecnológico

Lista todas las tecnologías, frameworks, librerías y herramientas utilizadas en el proyecto.

Ejemplo:

Categoría	Tecnología	Versión	Propósito
Lenguaje	Python	3.13	Lógica del juego completa
Librería de juego	Pygame	2.6.1	Ventana, gráficos, eventos y sonido
Control de versiones	Git	2.x	Manejo del código fuente
Repositorio remoto	GitHub	-	Almacenamiento y colaboración del código
Editor de código	VS Code	-	Desarrollo del proyecto

4.4. Diagrama de Clases y Modelo de Datos

Clases	Atributos	Métodos
Player	- Rect: dibuja al chef - Velocidad: el valor de la velocidad del chef - ingrediente: ingrediente que sostiene el chef - teclas: las teclas con las que puedes interactuar y mover al chef	- Mover: mueve al chef - Recoger: recoge un ingrediente de la estación - Soltar: deja un ingrediente en una estación - Interactuar: interactúa con las estaciones - update: recibe las teclas y llama al método mover - draw: dibuja al jugador
Ingredientes	- Nombre: el nombre del ingrediente - Estado: el estado en que se encuentra - Estación: la estación del ingrediente	Str: solo devuelve el nombre por ahora

Station	• Rect: dibuja la estación • Tipo: tipo de estación • ingrediente: es para detectar si tiene un ingrediente • timer: El valor para detectar cuánto tiempo estuvo un ingrediente • stock: Para que el almacén dispense ingredientes • tipo_ingrediente: Para que la estación almacén dispense un ingrediente en específico. • tiempo_min: Es tiempo mínimo que debe estar un ingrediente en la estación • tiempo_max: Es el tiempo máximo que debe estar un ingrediente	• Recibir: recibe un ingrediente del jugador • Entregar: entrega el ingrediente de la estación • update: actualiza el estado de la estación añadiendo tiempo y el estado del ingrediente • draw: dibuja la estación • Crear_platillo: metodo para recibir platillos.
Recipe	• nombre: Guarda el nombre de la receta • cocina: Guarda el nombre de la cocina • ingredientes: Guarda los ingredientes necesarios para la receta	• Str: retorna la receta
Order	• receta: elige aleatoriamente una receta de recipe • tiempo: el tiempo limite para entregar la orden • timer: el valor del contador • completada: para mostrar si la receta esta completada	• update: si la receta no esta completada incrementa el timer • esta_expirada: retorna cuando la receta esta expirada • completar: marca si la tarea esta completa • draw: dibuja la receta
Level	• tiempo: contiene la constante del tiempo	• tiempo agotado: detiene el tiempo

	del nivel(esta constante es para la duracion de todos los niveles) • puntos:guarda los puntos que consigue el jugador • timer: contador del nivel • num: valor para crear el almacén de los ingredientes • rcts: extrae las recetas de los platillos • estaciones: genera las estaciones necesarias	• update: incrementa el timer
Dish	• receta: el nombre de la receta • nombre: receta.nombre • ingredientes: ingredientes del platillo	
Game	• screen: crea la ventana del juego • clock: el reloj del juego • running: variable para mantener el juego activo • menu: cre el menú del juego • estado_actual: contiene el string "Menu" • chef1: creación de un objeto player • chef2: creación del segundo objeto player • level: crea el primer nivel • estaciones: guarda las estaciones del nivel • ordenes: guarda las ordenes •	• handle_event: Crea los eventos que se muestran en la ventana • update: Actualiza el estado del juego • draw: dibuja los objetos • entregar_platillos: añade puntos cuando se entrega un platillo • run: hace correr el juego

4.5. Diagrama de Flujo y Secuencia

Jugador	Presiona tecla de interactuar
Game.handle_event	Detecta la tecla presionada
Chef.interactuar(estaciones)	Recorre la lista de estaciones
colliderect()	Verifica si el chef toca alguna estación
chef.ingrediente is None	Decide si el chef debe recoger o soltar
Chef.recoger()	Si no carga nada, llama a Station.entregar()
Chef.soltar()	Si ya carga algo, llama a Station.recibir()
Station.entregar()	Vacía la estación y devuelve el ingrediente al chef
Station.recibir()	Valida si el ingrediente corresponde al tipo de estación
Loop principal	Continúa con el siguiente frame del juego

4.6. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Estructura de cocinas	Crear una clase distinta por cada cocina vs. una clase level reutilizable	Una clase level con datos distintos por número	Evita duplicar código; las 3 cocinas comparten la misma lógica, solo cambian datos
Validación de ingredientes en estaciones	Validar en el chef vs. validar en la estación	Validar en la estación	La estación conoce su propio tipo, mantiene la responsabilidad bien dividida (cada clase valida lo que le compete)

5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

OvercookTEC/	
├── main.py	→ Punto de entrada del programa
├── game.py	→ Game loop principal, coordina todo
├── constants.py	→ Constantes, teclas, diccionario de recetas
├── test.py	→ Pruebas manuales de las clases
├── entities/	
│ ├── __init__.py	
│ ├── player.py	→ Clase Chef, movimiento e interacción
│ ├── ingrediente.py	→ Clase Ingrediente
│ ├── station.py	→ Clase Station, estaciones de cocina
│ ├── recipe.py	→ Clase Recipe, define las recetas
│ └── order.py	→ Clase Order, pedidos con timer
├── levels/	
│ ├── __init__.py	
│ └── level.py	→ Clase Level, define niveles y sus estaciones
├── ui/	
│ ├── __init__.py	
│ ├── hud.py	→ Interfaz de puntaje, vidas, timer (de tu compañera)
│ └── order_panel.py	→ Panel de órdenes activas (de tu compañera)
├── requirements.txt	→ Dependencias del proyecto
├── .gitignore	
└── README.md	

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Entorno:

- Sistema Operativo: Windows 10+
 - Lenguaje: Python 3.13.5
 - Librería: Pygame 2.6.1
- Guía de instalación y ejecución:

```
# 1. Clonar el repositorio
git clone https://github.com/fabrigal509-eng/OvercookTEC.git
# 2. Entrar a la carpeta del proyecto
cd OvercookTEC
# 3. Instalar dependencias
pip install -r requirements.txt
# 4. Ejecutar el juego
python main.py
```

5.3. Estrategias y Casos de Prueba

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación aislada de funciones lógicas	Pytest	$\geq 75\%$ de las clases lógicas
Pruebas Funcionales (E2E)	Validación de las funcionalidades del juego, cocinar, cortar, emplatar.	pygame.event.post	100% de las recetas completadas
Pruebas de Rendimiento	Monitoreo de consumo de recursos	cProfile	Mantener Fps estables
Pruebas de integración	Probar relacionarse entre modulos independientes	Pytest	Comprobación del 100% de las estaciones

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	Jugar primer nivel	1. Clic botón jugar 2. Jugar el nivel	Cambio de pantalla al siguiente nivel	Se cambio al nivel 2	✓ Pasó
CP-002	Poner ingrediente en estación incorrecta	1. Tomar un ingrediente 2. Ponerlo en la estación incorrecta	No permitirlo	No se permitió	✓ Pasó
CP-003	Cambio de chefs	1. Usar WASD para el primer chef 2. Usar flechas para el segundo	Movimiento libre entre ambos	Movimiento libre	✓ Pasó

5.4. Resultados de pruebas y problemas encontrados

Resume los resultados de las pruebas ejecutadas.

Resultados:

Casos de prueba exitosos:

- Inicialización de segundo nivel a partir del primero
- Procesar un ingrediente a la vez en las estaciones
- Movimiento correcto de los dos chefs

Problemas encontrados:

- Problemas Solucionados:

- ☐ Las estaciones seguían cocinando y cortando aún sin ingrediente, se generó por que el timer nunca se detenía, entonces al poner otro ingrediente se quemaba o cortaba al segundo. Para arreglarlo se modificó el update de una las estaciones para que el incremento del temporizador se congelara cuando el ingrediente cambia de estado en el diccionario.
- ☐ El juego crashea cuando se intentan combinar ingredientes para hacer una receta, esto porque se trataban los ingredientes tanto como strings y objetos en diferentes archivos, causando el crash. Se soluciono modificando todos los strings para solo permitir objetos previamente declarados en un diccionario en constants.
- ☐ El juego se cerraba de golpe al intentar tomar un ingrediente, este error se generaba por errores de nomenclatura en los sprites de los ingredientes, python al no encontrar el archivo cerraba por completo el juego. Se soluciono añadiendo bloques de try y except, para en caso de no encontrar la imagen, cargara una estacion de color, para que no cayera el juego, luego se arreglo el problema de las nomenclaturas.

- Problemas no solucionados:

- ☐ Intento de bloques estilo inventario para una receta en construccion, se pretendía crear cuadros de color gris que indicaran los ingrredientes en un plato que se podrian usar en la receta. Se intentó esta funcionalidad pero los cuadros siempre aparecen enfrente de los sprites de los ingredientes, por consecuencia no se veían, al final se decidió dejar la idea de lado y concentrarse en otras partes del código.

6. Declaración de uso de Inteligencia Artificial y herramientas

6.1. Herramientas de IA utilizadas

Lista todas las herramientas de IA utilizadas durante el desarrollo. Sé específico con el nombre, versión (si aplica) y propósito.

Ejemplo:

Herramienta de IA	Tipo	Período de Uso	Propósito
ChatGPT / GPT-4	Modelo de lenguaje (LLM)	06/2026	Generación de ideas
GitHub Copilot	Asistente de código	06/2026	Autocompletado y sugerencias de código durante el desarrollo.
Claude	Modelo de lenguaje (LLM)	06/2026	Revisión de documentación y ayuda con la depuración.
Gemini	Modelo de lenguaje (LLM)	06/2026	Sugerencia de ideas y errores de código

6.2. Descripción del Uso de Inteligencia Artificial

Se utilizó la inteligencia artificial GitHub Copilot como autocompletado del texto, esto ayudó a saber como continuar con la programación de las funciones del código, aunque al final siempre era necesario revisar que era lo que sugería la IA.

Se utilizó ChatGPT de la empresa OpenAI para la planificación del proyecto, con ayuda de esta IA se logró planificar las clases, atributos, métodos e incluso el uso de github y la división del código en varios archivos para su mejor organización.

Se utilizó la inteligencia artificial Claude para la revisión del código en busca de errores y en la ayuda de depurar el proyecto. Fue de gran utilidad a la hora de solucionar fallos catastróficos que sucedieron durante el desarrollo.

Se empleó el uso de Gemini con el fin de buscar errores de código y buscar sugerencias para evitar sobrecargar los archivos con iteraciones absurdas.

6.3. Declaración de Responsabilidad y Autoría

Esta declaración debe ser redactada y firmada por todos los integrantes del equipo. Afirma que el trabajo final es producto del esfuerzo intelectual del equipo y que la IA fue utilizada como herramienta de apoyo.

Ejemplo:

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "OvercookTEC", declaramos que:

- El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
- Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
- Todo el contenido generado por IA fue revisado, comprendido, validado, y cuando fue necesario, corregido por los integrantes del equipo.
- Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
- El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Fabrizio Guillén Acevedo
Abigail Saborio

7. Conclusión

El desarrollo de OvercookTEC permitió cumplir satisfactoriamente con los objetivos planteados al inicio del proyecto. Se logró construir un juego funcional inspirado en Overcooked, implementando toda la lógica necesaria para que dos jugadores puedan moverse, recoger ingredientes, procesarlos en distintas estaciones de cocina y completar recetas dentro de un tiempo límite, replicando la dinámica de presión y trabajo en equipo característica del juego original.

El mayor reto del proyecto fue la organización del código en clases bien definidas. Inicialmente costó identificar qué responsabilidades debía tener cada clase y cómo debían comunicarse entre sí sin generar dependencias innecesarias. Sin embargo, planificar la estructura antes de programar —definiendo atributos y métodos de cada clase como Chef, Station, Ingrediente, Recipe, Order y Level— resultó clave para mantener un código ordenado, fácil de depurar y de extender conforme avanzaba el desarrollo.

El trabajo en equipo también representó un aprendizaje importante. Dividir las responsabilidades entre la lógica del juego y la interfaz gráfica, y coordinar los cambios mediante Git con ramas independientes, permitió que ambos integrantes avancen en paralelo sin generar conflictos significativos en el código compartido.

Como mejora a futuro, en proyectos similares sería valioso definir desde el inicio una documentación clara de los métodos públicos de cada clase, de manera que la integración entre lógica e interfaz sea aún más fluida desde las primeras etapas del desarrollo, reduciendo el tiempo de ajuste cuando ambas partes convergen.